

Transaction Processing, Concurrency Control, and Lost Update Problem

1 Introduction

A **transaction** is a logical unit of database processing consisting of a sequence of operations such as:

- `read_item(X)`
- `write_item(X)`

Transaction processing systems involve:

- large shared databases
- hundreds of concurrent users
- fast response time

Examples: banking, airline reservations, e-commerce, credit card processing.

2 ACID Properties of Transactions

A reliable transaction processing system must ensure that every transaction behaves correctly even in the presence of failures or concurrent execution. To guarantee correctness, a transaction must satisfy the **ACID** properties: Atomicity, Consistency, Isolation, and Durability.

2.1 Atomicity

Atomicity requires that a transaction be treated as a single, indivisible unit of work. Either *all* its operations are executed successfully and the transaction commits, or *none* of them take effect (the transaction aborts).

Example

Consider a banking transaction transferring Rs. 5000 from Account A to Account B:

1. Read(A)
2. A := A - 5000
3. Write(A)
4. Read(B)
5. B := B + 5000
6. Write(B)

If the system crashes after step 3 but before step 6, without atomicity A would lose Rs. 5000 permanently. Atomicity ensures that either:

- all 6 steps succeed (commit), or
- every change is undone (rollback).

2.2 Consistency

A transaction must take the database from one *consistent* state to another. It cannot violate integrity constraints such as:

- Primary key / foreign key constraints
- Balance never goes negative
- Total seat reservations $\leq$ total airplane capacity
- Domain constraints (e.g., age cannot be negative)

Example

If an airline flight has a capacity of 180 seats, and a transaction attempts to book 200 seats, the system should reject it. A consistent database never allows integrity rules to be broken.

2.3 Isolation

Transactions running concurrently should not interfere with one another. Even though operations may actually interleave, each transaction must behave *as if it executed alone in isolation*.

Isolation prevents problems such as:

- Lost update
- Dirty read
- Unrepeatable read
- Incorrect summary

Example

If T_1 updates a bank balance and T_2 reads it at the same time, isolation ensures that T_2 either:

- sees the value before T_1 started, or
- sees the value after T_1 commits,

but never a mixture of partial updates.

2.4 Durability

Once a transaction commits, its changes must become permanent. Even if the system crashes immediately after the commit, the updates must survive.

Durability ensures that:

- committed updates are written to non-volatile storage,
- recovery techniques restore the committed state after failures.

Example

If a transaction successfully transfers money and commits, a crash cannot undo it. After restart, the updated balances must appear exactly as the committed state.

3 ACID Properties with Real Database Example

To understand ACID properties, consider a real **Banking System** database where users can transfer money between accounts.

3.1 Database Schema

Accounts Table

account_id	holder_name	balance
101	Alice	5000
102	Bob	3000

Table 1: Accounts Table

Transactions Table

transaction_id	from_acc	to_acc	amount	status
1	101	102	1000	pending

Table 2: Transactions Table

We now analyze how a simple transfer of **Rs. 1000 from Alice to Bob** satisfies ACID.

3.2 Atomicity

Atomicity ensures that a transaction is **all or nothing**. Either:

- Alice is debited Rs. 1000 and Bob is credited Rs. 1000, or
- Neither operation happens.

Transaction Script:

```
BEGIN TRANSACTION;  
UPDATE Accounts SET balance = balance - 1000 WHERE account_id = 101;  
UPDATE Accounts SET balance = balance + 1000 WHERE account_id = 102;  
COMMIT;
```

If any statement fails:

```
ROLLBACK;
```

This prevents half-completed money transfers.

3.3 Consistency

Consistency ensures that database rules remain valid before and after a transaction.

Before transfer:

$$Alice = 5000, \quad Bob = 3000, \quad Total = 8000$$

After successful transfer:

$$Alice = 4000, \quad Bob = 4000, \quad Total = 8000$$

The **total balance remains unchanged**, meaning database constraints and rules are preserved.

If an incorrect state occurs, such as:

$$Alice = 4000, \quad Bob = 3000, \quad Total = 7000,$$

consistency is violated — but ACID prevents this.

3.4 Isolation

Isolation ensures that concurrent transactions do not interfere with each other.

Consider two concurrent transactions:

- T1: Alice $\rightarrow$ Bob (1000)
- T2: Alice $\rightarrow$ Charlie (2000)

Without isolation, both may read Alice's balance as 5000 and produce:

$$5000 - 1000 - 2000 = \text{Incorrect}$$

This leads to lost updates.

Using proper isolation levels (e.g., Serializable), each transaction sees a consistent view, preventing:

- Dirty reads
- Non-repeatable reads
- Phantom reads

Thus, the transactions execute independently.

3.5 Durability

Durability ensures that once a transaction commits, its effect is **permanent**, even if:

- The system crashes
- Power fails
- The database restarts

After:

`COMMIT;`

The updated balances:

$$Alice = 4000, \quad Bob = 4000$$

are safely stored using mechanisms like:

- Write-Ahead Logging (WAL)
- Redo logs
- Disk persistence

No committed transaction is ever lost.

3.6 Summary of ACID

- **Atomicity:** Transfer completes fully or not at all.
- **Consistency:** Total balance before = total balance after.
- **Isolation:** Parallel transfers do not interfere.
- **Durability:** Once committed, the transfer survives crashes.

4 Read and Write Operations

4.1 read_item(X)

1. Locate disk block containing X
2. Copy block into DB buffer
3. Copy item X into program variable

4.2 write_item(X)

1. Locate disk block containing X
2. Copy block into buffer
3. Write program variable X to buffer
4. Output modified block to disk

5 Example Transactions

Transaction T_1

Transfers N seats from flight X to flight Y:

```
read_item(X);  
X := X - N;  
write_item(X);  
read_item(Y);  
Y := Y + N;  
write_item(Y);
```

Transaction T_2

Reserves M seats on flight X :

```
read_item(X);  
X := X + M;  
write_item(X);
```

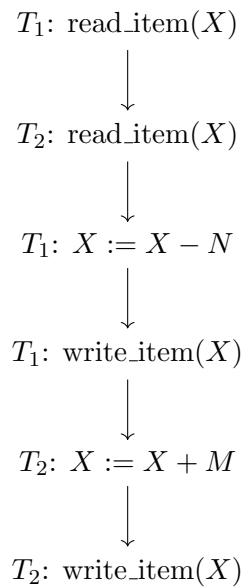
6 Lost Update Problem

6.1 Definition

The **Lost Update Problem** occurs when:

Two transactions read the same data and update it, but one update overwrites the other, causing one result to be **lost**.

6.2 Interleaving Causing the Problem



6.3 Numerical Example

Let initial value:

$$X = 80, \quad N = 5, \quad M = 4$$

Correct final value:

$$X = 80 - 5 + 4 = 79$$

Actual final value due to lost update:

$$X = 84$$

Reason:

- T_2 reads old value of X before T_1 updates it.
- T_1 writes 75 but then T_2 overwrites X with 84.

7 Temporary Update / Dirty Read / Uncommitted Dependency Problem

The **Temporary Update Problem**, also known as the **Dirty Read Problem**, occurs when a transaction updates a data item but then fails before committing. Meanwhile, another transaction reads the updated value even though the update may later be rolled back.

Explanation

- Transaction T_1 updates a data item X .
- Before T_1 commits, transaction T_1 fails.
- Because T_1 failed, the system must **roll back** the update to restore X to its original value.
- However, before the rollback occurs, another transaction T_2 reads the **temporary, uncommitted value** of X .

The value read by T_2 is called **dirty data** because it was produced by a transaction that has not committed. Hence, this is known as the **Dirty Read Problem**.

Example

Assume the initial value of data item:

$$X = 50$$

Transaction Schedule

Step	Operation
1	T1: Read $X = 50$
2	T1: Write $X = 80$ (temporary value)
3	T2: Read $X = 80$ (dirty read)
4	T1: Fails (system crash or error)
5	System: Rollback X to 50

Table 3: Illustration of the Dirty Read Problem

Why This is a Problem?

- T2 reads a value (80) that **will never be committed**.
- The final database value is restored to 50 after rollback.
- T2 has made decisions based on an invalid, temporary value.

This leads to inconsistency in the system.

How to Prevent Dirty Reads?

Dirty reads occur in low isolation levels (e.g., `READ UNCOMMITTED`). They can be prevented by using stricter isolation levels:

- **READ COMMITTED**
- **REPEATABLE READ**
- **SERIALIZABLE**

These levels ensure that a transaction **cannot read uncommitted updates** from another transaction.

8 Incorrect Summary Problem (Inconsistent Analysis)

The **Incorrect Summary Problem** occurs when one transaction is computing an aggregate value (such as a sum), while another transaction is concurrently updating the same data items. Because the update is only partially applied when the summary transaction reads the data, the final computed result becomes inconsistent.

Example

Assume we have two data items:

$$X = X_0, \quad Y = Y_0$$

Transaction T2 transfers an amount N from X to Y :

$$X := X - N, \quad Y := Y + N$$

Transaction T3 computes the value:

$$\text{sum} = X + Y$$

Interleaving Schedule

Step	Operation
1	T2: read_item(X)
2	T2: $X := X - N$
3	T2: write_item(X)
4	T3: read_item(X) (reads decreased value)
5	T2: read_item(Y)
6	T2: $Y := Y + N$
7	T3: read_item(Y) (reads original value)
8	T2: write_item(Y)
9	T3: $\text{sum} := X + Y$

Table 4: Schedule Causing Incorrect Summary

Explanation

T3 reads:

- X **after** N has been subtracted.
- Y **before** N has been added.

Therefore:

$$\text{sum} = (X_0 - N) + Y_0 = (X_0 + Y_0) - N$$

The summary is **incorrect by exactly** N .

Why This Happens

This occurs because T3 sees a **partial update** made by T2. The transfer (subtract from X and add to Y) is only half complete when T3 computes the sum.

How to Prevent the Incorrect Summary Problem

Use stronger isolation techniques:

- **Strict Two-Phase Locking (Strict 2PL)**
- **Serializable isolation level**
- **Locking both X and Y before computing the sum**

These methods ensure that T3 cannot read the intermediate state of T2.

9 Unrepeatable Read Problem

An **unrepeatable read** occurs when a transaction reads the same data item twice and obtains different values because another transaction modified the item between the two reads. This violates the isolation property of ACID.

Example Scenario: Airline Reservation

Consider a table `flights`:

flight_no	seats_available
PK301	5

Let two transactions execute concurrently:

- T_1 : Customer inquiry checking seat availability.
- T_2 : Another booking being processed.

Schedule Demonstrating the Problem

```
T1: read_item(seats_available);    // returns 5

T2: read_item(seats_available);    // returns 5
T2: seats_available := seats_available - 1;  // now 4
T2: write_item(seats_available);    // database updated to 4

T1: read_item(seats_available);    // now returns 4
```

Transaction T_1 reads the value of `seats_available` twice, but the values differ (5 and 4) because T_2 updated the item in between the two reads. This results in an **unrepeatable read**.

10 The Phantom Read Problem

A **phantom read** occurs when a transaction reads a set of rows satisfying a certain condition, but when it repeats the query, it finds additional rows (phantoms) that were inserted or deleted by another transaction in the meantime.

Example 1: Employee Scenario

Consider the following two transactions:

- Transaction **T1** executes:

```
SELECT * FROM employees WHERE department_id = 10;
```

It returns 5 rows.

- While **T1** is active, transaction **T2** inserts a new employee:

```
INSERT INTO employees (id, name, department_id)
VALUES (101, 'Alice', 10);
```

- When **T1** executes the same query again:

```
SELECT * FROM employees WHERE department_id = 10;
```

It now returns 6 rows. The additional row is the *phantom*.

Example 2: Student–Course Scenario

Consider a university database with a `student_courses` table:

- Transaction **T1** executes:

```
SELECT * FROM student_courses WHERE course_id = 101;
```

It returns 30 students enrolled in course 101.

- While **T1** is active, transaction **T2** deletes a student who withdraws:

```
DELETE FROM student_courses
WHERE student_id = 205 AND course_id = 101;
```

- When **T1** executes the same query again:

```
SELECT * FROM student_courses WHERE course_id = 101;
```

It may now return 29 students. The deleted student is a *phantom disappearance*.

10.1 Key Points

- Phantoms involve **new rows appearing** or existing rows disappearing in the result set.
- Different from non-repeatable reads, which involve changes to existing rows' values.
- Prevented by **Serializable isolation level**.
- Lower isolation levels, such as Read Committed or Repeatable Read, do not prevent phantom reads.

11 Why Recovery Is Needed

Whenever a transaction is submitted to a DBMS for execution, the system is responsible for ensuring that either:

- All the operations in the transaction are completed successfully and their effects are recorded permanently in the database (**committed**), or
- The transaction has no effect on the database or any other transactions (**aborted**).

The DBMS must not permit some operations of a transaction T to be applied to the database while other operations of T are not, because the transaction is a *logical unit of database processing*. If a transaction fails after executing some operations but before completing all of them, the operations already executed must be undone so they have no lasting effect.

11.1 Types of Failures

Failures are generally classified as **transaction, system, and media failures**. Common reasons for transaction failure include:

1. **Computer failure (system crash):** Hardware, software, or network errors may occur during transaction execution. Hardware crashes are often media failures (e.g., main memory failure).

2. **Transaction or system error:** Operations such as integer overflow, division by zero, erroneous parameters, or logical programming errors can cause failure. Users may also interrupt a transaction manually.
3. **Local errors or exception conditions:** Certain conditions may necessitate cancellation of the transaction, such as missing data. Exception conditions, like insufficient account balance in a banking transaction, may cancel the transaction; these do not count as transaction failures if properly programmed.
4. **Concurrency control enforcement:** The concurrency control mechanism may abort a transaction if it violates serializability or to resolve deadlocks among multiple transactions. Aborted transactions due to these reasons are typically restarted automatically.
5. **Disk failure:** Data loss may occur due to read/write malfunctions or disk crashes during transaction operations.
6. **Physical problems and catastrophes:** Includes power or air-conditioning failures, fire, theft, sabotage, accidental overwriting of disks or tapes, or mounting the wrong tape.

Failures of types 1–4 are more common than types 5 or 6. For failures of types 1–4, the system must maintain sufficient information to allow quick recovery. Failures of types 5 or 6 are less frequent but require more extensive recovery efforts.

11.2 Importance of Transactions in Recovery

The concept of a transaction is fundamental to techniques for:

- **Concurrency control**, ensuring multiple transactions execute correctly together.
- **Recovery from failures**, ensuring the database remains consistent even after a crash or error.

12 Transaction and System Concepts

This section discusses additional concepts relevant to transaction processing.

12.1 Transaction States and Additional Operations

A transaction is an **atomic unit of work** that should either be completed entirely or not at all. For recovery purposes, the system needs to track when each transaction starts, terminates, commits, or aborts.

The recovery manager of the DBMS tracks the following operations:

- **BEGIN_TRANSACTION:** Marks the start of a transaction.
- **READ or WRITE:** Specifies read or write operations on database items executed as part of the transaction.
- **END_TRANSACTION:** Marks the end of transaction operations. Additional checks may be performed to see if the transaction can be committed.
- **COMMIT_TRANSACTION:** Signals a successful end of the transaction. Changes executed by the transaction are permanently applied to the database.
- **ROLLBACK (or ABORT):** Signals an unsuccessful end. All changes applied by the transaction must be undone.

12.1.1 Transaction States

Figure 1 illustrates the state transitions of a transaction:

- **Active:** Immediately after a transaction starts. READ and WRITE operations can be executed.
- **Partially Committed:** After END_TRANSACTION, checks are performed to ensure the transaction can commit. Some recovery protocols ensure system failures do not prevent permanent recording of changes.
- **Committed:** Transaction successfully completed; all changes must be permanent.
- **Failed:** Occurs if a check fails or the transaction is aborted during the active state. Requires rollback of WRITE operations.
- **Terminated:** Transaction leaves the system; its information is removed from system tables. Failed or aborted transactions may be restarted later.

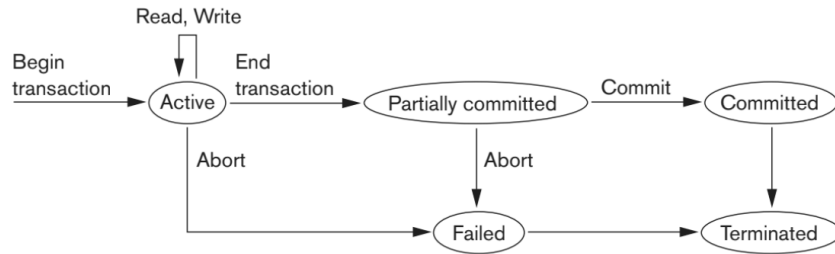


Figure 20.4
State transition diagram illustrating the states for transaction execution.

Figure 1: State Transition Diagram for Transaction Execution

13 System Log and Commit Point

13.1 The System Log

To ensure that a database can recover from failures that affect transaction execution, the DBMS maintains a **system log**. The log is a sequential, append-only file stored on disk so that it survives all failures except catastrophic disk failures.

Certain portions of the log are kept temporarily in **log buffers** in main memory. When the buffer becomes full, or when specific conditions occur, the buffer is force-written to the end of the disk log file. Additionally, disk log files are periodically backed up to archival storage to protect against catastrophic failures.

Types of Log Records

A **log record** describes an action performed by a transaction. Each transaction is assigned a unique transaction identifier T . The major log record types include:

1. [**start_transaction**, T] Indicates that transaction T has begun execution.
2. [**write_item**, T , X , **old_value**, **new_value**] Shows that transaction T changed database item X from **old_value** to **new_value**.

3. [**read_item**, **T**, **X**] Indicates that transaction T read the value of item X . (Many practical recovery protocols do not record READs unless needed for auditing.)
4. [**commit**, **T**] Denotes that transaction T completed successfully and its effects may be permanently recorded.
5. [**abort**, **T**] Shows that transaction T has been rolled back due to failure or explicit abort.

Undo and Redo Using Log Records

The log enables two essential recovery actions:

- **Undo:** If a transaction T fails, the DBMS traces the log backward and restores all updated items to their `old_value`.
- **Redo:** If T 's updates are in the log but not yet reflected on disk (due to buffering), the system re-applies all operations using `new_value`.

Thus, maintaining detailed `write_item` records ensures correct recovery even if a system crash occurs mid-transaction.

13.2 Commit Point of a Transaction

A transaction T reaches its **commit point** when:

- all its database operations have been executed successfully, and
- all the effects of the transaction have been recorded in the log.

Once the commit point is reached:

- the transaction writes a [**commit**, **T**] record into the log,
- the transaction is considered **committed**, and
- its effects must *permanently* be reflected in the database.

Force-Writing the Log

Because main memory may be lost during a system crash, the DBMS ensures durability using a technique called **force-writing**:

Before a transaction commits, all log records for that transaction must be written to the disk log file.

This guarantees that no committed transaction is lost during recovery.

13.3 Buffer Replacement Policies in DBMS

To manage limited memory, the DBMS must decide which pages to evict when new pages are loaded from disk. Several specialized replacement strategies include:

1. Domain Separation (DS) Method

The DBMS cache is divided into domains, each dedicated to a page type (data, index, log pages, etc.). Each domain uses LRU internally. Although DS improves performance, it is static and does not adapt to dynamic workloads. Variants such as **GRU (Group LRU)** assign priorities and evict from the lowest-priority domain first.

2. Hot Set Method

Useful for queries requiring repeated scans—such as nested-loop joins. A **hot set** is a group of pages that will be accessed repeatedly. These pages are kept in memory until processing completes, avoiding unnecessary replacement.

3. DBMIN Method

DBMIN uses the **Query Locality Set Model (QLSM)** to estimate how many buffers each file instance in a query will need. Each file instance gets a **locality set**, similar to the working set concept in operating systems. Buffers are allocated dynamically based on predicted page access patterns for the query type (e.g., SELECT, JOIN).

14 Desirable Properties of Transactions: Isolation Levels

The Desirable Properties of Transactions are ACID which are already discussed earlier in detail. Now let's move onto the detail of isolation levels. Isolation determines how much one transaction is protected from the effects of other concurrently running transactions. The following subsections describe Levels 0 to 3, along with Snapshot Isolation, their properties, and examples.

14.1 Isolation Level 0

Prevents:

- A transaction cannot overwrite **dirty** values written by a higher-level transaction.

Allows:

- Dirty reads
- Unrepeatable reads
- Phantom reads
- Lost updates (possible)

Example:

- Initial value: $X = 100$
- T_1 : writes $X = 200$ (uncommitted)
- T_2 : reads $X = 200$ (dirty read allowed)

Level 0 only prevents T_2 from writing a new value over T_1 's uncommitted change.

14.2 Isolation Level 1 (No Lost Updates)

Prevents:

- Lost updates

Allows:

- Dirty reads
- Unrepeatable reads
- Phantom reads

Example (Lost Update Prevention):

- Initial value: $X = 100$
- T_1 : reads $X = 100$
- T_2 : reads $X = 100$
- T_1 : writes $X = 90$
- T_2 : tries to write $X = 80 \rightarrow$ blocked or delayed

Thus T_2 cannot overwrite T_1 's update before commit.

14.3 Isolation Level 2 (No Lost Updates + No Dirty Reads)

Prevents:

- Lost updates
- Dirty reads

Allows:

- Unrepeatable reads
- Phantom reads

Example (Dirty Read Prevention):

- Initial value: $X = 500$
- T_1 : writes $X = 300$ (not committed)
- T_2 : attempts "READ X" but is blocked

14.4 Isolation Level 3 (True Isolation: Repeatable Reads)

This is the strongest classical ANSI isolation level.

Prevents:

- Lost updates
- Dirty reads
- Unrepeatable reads

Allows:

- Phantom reads

Example (Repeatable Read Guarantee):

Initial table:

Products:

ID	Price
1	100
2	200

- T_1 : SELECT Price WHERE ID = 1 $\rightarrow$ 100
- T_2 : UPDATE Product SET Price = 150 WHERE ID = 1; COMMIT;
- T_1 : SELECT Price WHERE ID = 1 $\rightarrow$ still 100 (repeatable)

Example (Phantom Problem):

- T_1 : SELECT * FROM Products WHERE Price $\geq$ 100 (returns rows 1 and 2)
- T_2 : INSERT Product(3, 120)
- T_1 : same query returns rows 1, 2, and 3 (phantom)

14.5 Snapshot Isolation

Snapshot Isolation is typically implemented using Multi-Version Concurrency Control (MVCC). Each transaction reads from a **snapshot** of committed data at the time the transaction begins.

Prevents:

- Dirty reads
- Unrepeatable reads
- Phantom reads

Allows:

- Some write anomalies (e.g., write skew)

How Snapshot Isolation Works:

- Reads always come from the transaction's snapshot.
- Writes create new versions of data items.
- A transaction may commit only if no other concurrent transaction has modified the same data.

Example:

Initial state:

$$A = 100, \quad B = 100$$

Both T_1 and T_2 start and read the same snapshot.

- T_1 : UPDATE $A = 80$
- T_2 : UPDATE $B = 120$

Both commit successfully because they update different items and do not conflict.

Phantom Prevention:

- T_1 : SELECT * FROM Orders WHERE amount > 100; (snapshot contains 10 rows)
- T_2 : INSERT new order with amount = 200; COMMIT;
- T_1 : same SELECT returns the same 10 rows (no phantom)

14.6 Summary Table

Level	Dirty Reads	Lost Updates	Unrepeatable Reads	Phantoms
0	Allowed	Possible	Allowed	Allowed
1	Allowed	Prevented	Allowed	Allowed
2	Prevented	Prevented	Allowed	Allowed
3	Prevented	Prevented	Prevented	Allowed
Snapshot	Prevented	Via conflict detection	Prevented	Prevented

15 Why Concurrency Control?

Without concurrency control:

- database becomes inconsistent
- updates are lost
- dirty data is read
- summaries become incorrect

Thus, concurrency control ensures:

- serializability
- isolation
- correctness

16 Characterizing Schedules Based on Recoverability

16.1 Schedules (Histories) of Transactions

A **schedule** (or history) S of transactions $T_1, T_2, \dots, T_n$ is a sequence that represents the order in which the operations of these transactions are executed. Operations from different transactions may be interleaved, but the

operations within each individual transaction must appear in the same order as defined in that transaction.

The operations of interest are:

$$b_i, r_i(X), w_i(X), e_i, c_i, a_i$$

representing begin, read, write, end, commit, and abort for transaction T_i .

Example schedule:

$$S_a : r_1(X); r_2(X); w_1(X); r_1(Y); w_2(X); w_1(Y)$$

16.2 Conflicting Operations

Two operations conflict if:

1. They belong to different transactions.
2. They access the same data item X .
3. At least one of them is a write operation.

Examples of conflicts:

$$r_1(X) \text{ and } w_2(X), \quad r_2(X) \text{ and } w_1(X), \quad w_1(X) \text{ and } w_2(X)$$

Non-conflicting examples:

$$r_1(X) \text{ and } r_2(X), \quad w_2(X) \text{ and } w_1(Y)$$

16.3 Complete Schedules

A schedule S is **complete** if:

1. It contains all operations of all participating transactions, including their final commit or abort.
2. The order of operations within each transaction follows the transaction definition.
3. Conflicting operations are totally ordered.

16.4 Committed Projection

The **committed projection** $C(S)$ of a schedule S includes only the operations of transactions that commit in S .

16.5 Recoverable Schedules

A schedule S is **recoverable** if a transaction T does not commit until all transactions T' from which it has read have committed.

If T reads item X written by T' , then:

T' must commit before T .

Example of a recoverable schedule:

$S'_a : r_1(X); r_2(X); w_1(X); r_1(Y); w_2(X); c_2; w_1(Y); c_1$

Example of a non-recoverable schedule:

$S_c : r_1(X); w_1(X); r_2(X); r_1(Y); w_2(X); c_2; a_1$

Here, T_2 reads X from T_1 but commits before T_1 commits. If T_1 aborts, T_2 must also abort even though it committed earlier, violating recoverability.

16.6 Cascading Rollback and Cascadeless Schedules

A **cascading rollback** occurs when an uncommitted transaction must abort because it read data from another transaction that aborted.

A schedule is **cascadeless** if every transaction reads only from committed transactions:

T reads item $X \implies X$ was written by a committed transaction.

Thus, cascading rollbacks are avoided.

16.7 Strict Schedules

A schedule is **strict** if:

No transaction is allowed to read or write item X
until the last transaction that wrote X has committed or aborted.

Example of a non-strict schedule:

$$S_f : w_1(X, 5); w_2(X, 8); a_1$$

If T_1 aborts, restoring the before-image of X (say, 9) is incorrect because T_2 already overwrote X with 8.

Strict schedules avoid this problem and greatly simplify recovery because:

Undoing a write restores the correct before image.

16.8 Relationship Among Schedule Types

The hierarchy of schedules is:

$$\text{Strict} \subset \text{Cascadeless} \subset \text{Recoverable}$$

Thus:

- Every strict schedule is cascadeless.
- Every cascadeless schedule is recoverable.

Most recovery protocols allow only **strict schedules**.

Definitions

Schedule

A **schedule** is a sequence of operations from a set of concurrent transactions that preserves the order of operations within each individual transaction.

Serial Schedule

A **serial schedule** is a schedule in which the operations of each transaction are executed consecutively, without any interleaving of operations from other transactions.

Non-Serial Schedule

A **non-serial schedule** is a schedule in which operations from multiple concurrent transactions are interleaved.

Serializability

Serializability refers to identifying non-serial schedules that allow concurrent execution of transactions without interference, yet produce a final database state that could also be produced by some serial execution.

i.e. (in essence)

- When a set of transactions executes concurrently,
- A non-serial schedule is considered **correct**
- If it produces the same result as some serial execution.

Such a schedule is called **serializable**.

To prevent inconsistencies caused by interference among transactions, it is essential to guarantee the **serializability** of concurrent transactions.

Characterizing Schedules Based on Recoverability

Conflicting Operations

Two operations in a schedule are said to **conflict** if:

- They belong to different transactions.
- They access the same data item X .
- At least one of them is a `write_item(X)` operation.

Two operations conflict if changing their order leads to a different outcome.

Types of Conflicts:

- Read–Write conflict (RW)
- Write–Write conflict (WW)

Characterizing Schedules Based on Recoverability (cont'd.)

Recoverable Schedules

- Recovery from failures is possible.
- DBMS should not allow **nonrecoverable schedules**.
- No committed transaction should ever need to be rolled back.

Cascading Rollback

Occurs when an uncommitted transaction must roll back because it read a value written by another transaction that failed.

- Cascading rollback **may occur** in some recoverable schedules.

Characterizing Schedules Based on Recoverability (cont'd.)

Cascadeless Schedules

- Avoid cascading rollback.
- Every read operation reads only **committed** data.

Strict Schedules

- A transaction cannot read or write item X until the last transaction that wrote X has committed or aborted.
- Simplifies recovery.
- Recovery is done by restoring the **before image**.

20.5 Characterizing Schedules Based on Serializability

Serializable Schedules

- Considered **correct** when concurrent transactions execute.
- Represent concurrent transactions as if executed in series.
- Either T_1 before T_2 , or T_2 before T_1 .

Serializability (cont'd.)

Serial vs. Non-Serial

- Serial schedules restrict concurrency.
- Interleaving is often necessary in practice.

Solution: Determine which non-serial schedules are equivalent to a serial one.

A schedule of n transactions is **serializable** if it is equivalent to some serial schedule of the same n transactions.

Serializability (cont'd.)

Result-Equivalent Schedules

- Produce the same final database state.
- But may be accidental.
- Not sufficient to define logical equivalence.

Serializability (cont'd.)

Conflict Equivalence

Two schedules are conflict equivalent if:

- The relative order of every pair of conflicting operations is identical.

Conflict Serializable Schedule

Schedule S is **conflict serializable** if it is conflict equivalent to some serial schedule S' .

How Serializability is Used in Concurrency Control

- Being serializable is different from being serial.
- Serializable schedules allow concurrency without violating correctness.
- Hard to test in real time.
- DBMS uses protocols (rules) to ensure serializability.

View Equivalence and View Serializability

View Equivalence

Two schedules are view equivalent if:

- Every read in each schedule reads the result of the same write.
- The final writes produce the same result.

View Serializable

A schedule is view serializable if it is view equivalent to a serial schedule.

17 Examples of Serial and Non-Serial Schedules

Figure 2 illustrates four schedules involving two transactions T_1 and T_2 . The operations of these transactions read and write two data items X and Y . The figure compares:

- Two **serial schedules** (A and B)
- Two **non-serial schedules** (C and D)

17.1 Schedule A: Serial (T_1 then T_2)

Schedule A is a **serial schedule** where all operations of T_1 execute first, followed by all operations of T_2 .

$$T_1 \rightarrow T_2$$

Characteristics:

- No interleaving of instructions.
- T_2 reads the updated value of X produced by T_1 .
- Produces one possible correct serial database state.

The precedence graph contains a single edge:

$$T_1 \rightarrow T_2$$

so the schedule is serializable.

17.2 Schedule B: Serial (T_2 then T_1)

Schedule B is also a **serial schedule**, but the order is reversed:

$$T_2 \rightarrow T_1$$

Properties:

- Equivalent to executing the transactions one after another.

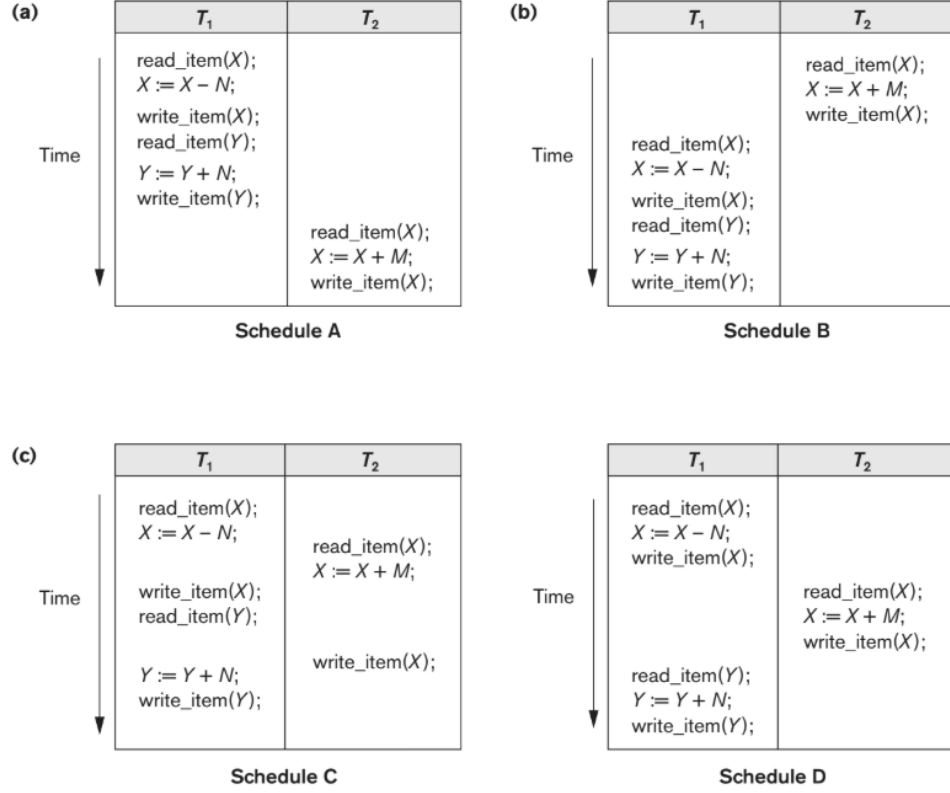


Figure 20.5

Examples of serial and nonserial schedules involving transactions T_1 and T_2 . (a) Serial schedule A: T_1 followed by T_2 . (b) Serial schedule B: T_2 followed by T_1 . (c) Two nonserial schedules C and D with interleaving of operations.

Figure 2: Examples of serial and non-serial schedules involving T_1 and T_2 . (a) Serial schedule A: T_1 followed by T_2 . (b) Serial schedule B: T_2 followed by T_1 . (c–d) Two non-serial schedules with interleaving of operations.

- T_1 reads values after T_2 finishes writing.

The precedence graph is:

$$T_2 \rightarrow T_1$$

again serializable by definition.

17.3 Schedule C: Non-Serial (Interleaving)

Schedule C interleaves operations of T_1 and T_2 .

Important observations:

- Both transactions access X .
- T_1 writes X before T_2 writes it, producing a conflict.
- T_2 writes X before T_1 writes Y .

This creates **two conflicts in opposite directions**:

$$T_1 \rightarrow T_2 \quad \text{and} \quad T_2 \rightarrow T_1$$

Thus the precedence graph contains a **cycle**, meaning:

Schedule C is **not serializable**.

17.4 Schedule D: Non-Serial but Serializable

Schedule D also interleaves operations but in a different order than C.

Properties:

- T_1 writes X before T_2 reads X , creating $T_1 \rightarrow T_2$.
- No conflicting operation exists in the reverse direction.

Thus the precedence graph has only one direction:

$$T_1 \rightarrow T_2$$

No cycles occur, meaning:

Schedule D is **conflict serializable**, equivalent to Schedule A.

17.5 Summary of the Four Schedules

- **Schedule A**: Serial, T_1 before T_2 (Serializable)
- **Schedule B**: Serial, T_2 before T_1 (Serializable)
- **Schedule C**: Non-serial with cycles (Not serializable)
- **Schedule D**: Non-serial but acyclic (Conflict serializable)

Thus, Schedules A, B, and D are correct serializable schedules, while Schedule C is not.

18 Testing for Serializability of a Schedule

There exists a straightforward algorithm for determining whether a given schedule is **conflict serializable**. In practice, most concurrency control methods do not explicitly test serializability. Instead, the DBMS enforces **protocols** that ensure that any schedule following these protocols will be serializable. Some methods guarantee serializability most of the time while reducing overhead, and may not guarantee it absolutely.

To understand these protocols, we study **Algorithm 20.1**, which tests conflict serializability by constructing a **precedence graph** (also called a **serialization graph**).

Precedence Graph Definition

Given a schedule involving transactions $T_1, T_2, \dots, T_n$, we construct a directed graph:

$$G = (N, E)$$

where:

- $N = \{T_1, T_2, \dots, T_n\}$ is the set of nodes, one for each transaction.
- $E = \{e_1, e_2, \dots, e_m\}$ is the set of directed edges.

An edge $(T_j \rightarrow T_k)$ is added if:

- T_j and T_k contain a pair of **conflicting operations**, and
- the conflicting operation of T_j occurs **before** that of T_k in the schedule.

Conflicts arise from:

read–write, write–read, write–write

Algorithm 20.1: Testing Conflict Serializability of a Schedule S

1. For each transaction T_i in S , create a node labeled T_i in the precedence graph.

2. For every case where T_j performs a `read_item(X)` after T_i performs a `write_item(X)`, add an edge:

$$(T_i \rightarrow T_j)$$

3. For every case where T_j performs a `write_item(X)` after T_i performs a `read_item(X)`, add an edge:

$$(T_i \rightarrow T_j)$$

4. For every case where T_j performs a `write_item(X)` after T_i performs a `write_item(X)`, add an edge:

$$(T_i \rightarrow T_j)$$

5. The schedule S is **conflict serializable** if and only if the precedence graph has **no cycles**.

Cycles and Serializability

A cycle in a directed graph is a sequence:

$$C = ((T_j \rightarrow T_k), (T_k \rightarrow T_p), \dots, (T_i \rightarrow T_j))$$

whose edges form a closed loop (the start and end node are the same).

- If the precedence graph contains a cycle, the schedule is **not serializable**.
- If there is **no cycle**, the schedule is **serializable**.

Equivalent Serial Schedule

If the precedence graph is acyclic, we may create an equivalent serial schedule S' as follows:

- Whenever an edge $(T_i \rightarrow T_j)$ exists, T_i must appear **before** T_j in the equivalent serial schedule.

Edges may optionally be labeled with the data item(s) that caused the conflict (Figure 20.7). These labels do not matter when checking for cycles.

Serializability Testing: Example (Figure 20.8)

Read/Write Operations of the Transactions (Figure 20.8a)

Transaction T_1	Transaction T_2	Transaction T_3
read_item(X);	read_item(Z);	read_item(Y);
write_item(X);	read_item(Y);	read_item(Z);
read_item(Y);	write_item(Y);	write_item(Y);
write_item(Y);	read_item(X);	write_item(Z);
	write_item(X);	

Figure 3: Read and write operations of transactions T_1 , T_2 , and T_3 .

The operations of each transaction are:

T_1 : read(X), write(X), read(Y), write(Y)

T_2 : read(Z), read(Y), write(Y), read(X), write(X)

T_3 : read(Y), read(Z), write(Y), write(Z)

These operations will interact differently depending on the schedule.

Schedule E (Figure 20.8b)

	Transaction T_1	Transaction T_2	Transaction T_3
Time ↓	read_item(X); write_item(X);	read_item(Z); read_item(Y); write_item(Y);	read_item(Y); read_item(Z);
	read_item(Y); write_item(Y);	read_item(X); write_item(X);	write_item(Y); write_item(Z);

Schedule E

Figure 4: Schedule E for T_1 , T_2 , and T_3 .

Schedule E interleaves the transactions in a way that produces multiple conflicts across X , Y , and Z . This creates cycles when constructing the precedence graph.

Result

Schedule E is **not serializable**.

Reason: Multiple conflict cycles exist, including:

$$X(T_1 \rightarrow T_2), \quad Y(T_2 \rightarrow T_1)$$

and a longer cycle involving all three transactions:

$$X(T_1 \rightarrow T_2), \quad YZ(T_2 \rightarrow T_3), \quad Y(T_3 \rightarrow T_1)$$

Since the precedence graph contains cycles, the schedule cannot be transformed into a serial one.

Precedence Graph for Schedule E (Figure 20.8d)

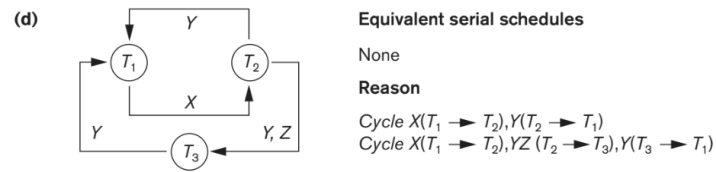


Figure 5: Precedence graph for Schedule E showing cycles.

Conclusion: No equivalent serial schedule exists.

Schedule F (Figure 20.8c)

	Transaction T_1	Transaction T_2	Transaction T_3
Time ↓	read_item(X); write_item(X);		read_item(Y); read_item(Z);
	read_item(Y); write_item(Y);	read_item(Z); read_item(Y); write_item(Y); read_item(X); write_item(X);	write_item(Y); write_item(Z);

Schedule F

Figure 6: Schedule F for T_1 , T_2 , and T_3 .

Schedule F interleaves the operations in a controlled way. Conflicts exist, but they do not produce a cycle.

Result

Schedule F is **conflict-serializable**.

An equivalent serial schedule must follow the direction of the edges in the precedence graph.

$$T_3 \rightarrow T_1 \rightarrow T_2$$

Precedence Graph for Schedule F (Figure 20.8e)

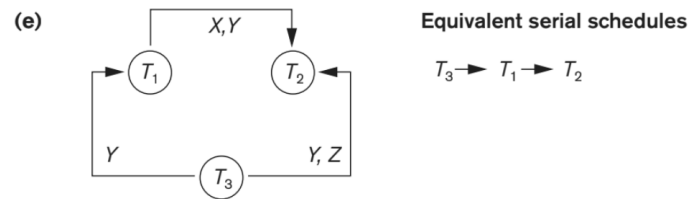


Figure 7: Precedence graph for Schedule F (acyclic).

Equivalent Serial Schedule: $T_3 \rightarrow T_1 \rightarrow T_2$

A Precedence Graph with Two Valid Serial Orders (Figure 20.8f)

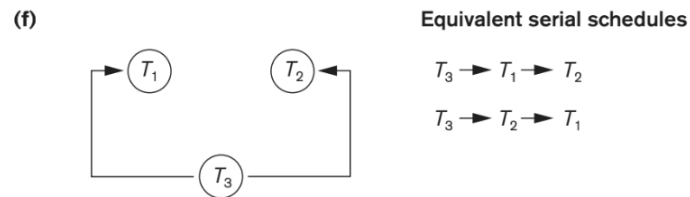


Figure 8: Precedence graph with two equivalent serial schedules.

This graph still has no cycles. Therefore, the schedule is serializable, and more than one serial order is possible:

$$T_3 \rightarrow T_1 \rightarrow T_2$$

$$T_3 \rightarrow T_2 \rightarrow T_1$$

The graph allows both because T_1 and T_2 do not constrain each other (no edge between them).